fastRhockey :: CHEAT SHEET

Tidy R access to hockey play-by-play, box scores, schedules, rosters and stats: the NHL Web API (api-web.nhle.com), NHL Stats REST, NHL EDGE tracking, NHL Records, ESPN and Fox NHL feeds, the PWHL plus AHL / OHL / WHL / QMJHL (HockeyTech), 13 European leagues, ESPN men's and women's college hockey, and season loaders from the SportsDataverse data releases: 880 exports. Sister of hoopR / wehoop / cfbfastR; Python mirror: `sportsdataverse.nhl` / `.pwhl` / `.hockey`. Page 1 of 2: NHL. Page 2: PWHL, HockeyTech, Europe, college, PHF.

v1.0.0.9000

R >= 4.1.0 · httr2 · dplyr · data.table
fastRhockey.sportsdataverse.org

GET STARTED

```
install.packages("fastRhockey")
# dev build
remotes::install_github(
  "sportsdataverse/fastRhockey")
library(fastRhockey)
library(dplyr)
```

Every function returns a tibble of class `fastRhockey_data` (or a named list of them). Live NHL wrappers hit `api-web.nhle.com/v1/` with no API key. Endpoint family overviews: `?nhl_web` `?nhl_stats_rest` `?nhl_edge` `?nhl_records` `?nhl_loaders` `?nhl_xg` `?pwhl`.

NHL RELEASE LOADERS

Pre-scraped full seasons (2010+) from the **sportsdataverse-data** GitHub releases (tag `nhl_*`). Default `seasons = most_recent_nhl_season()`; optional `dbConnection`, `tablename` write straight to a DB.

```
pbp  <- load_nhl_pbp(seasons = 2023:2025)
lite <- load_nhl_pbp_lite(2025)
  # slim columns
tb   <- load_nhl_team_box(2025)
pb   <- load_nhl_player_box(2025)
  # + skater / goalie
sch  <- load_nhl_schedule(2020:2025)
```

Loader	Release tag
`load_nhl_pbp` · `_pbp_full` · `_pbp_lite`	`nhl_pbp_full` / `nhl_pbp_lite`
`load_nhl_team_box` `_player_box` `_skater_box` `_goalie_box`	`nhl_*_boxscores`
`load_nhl_schedule` `_rosters` `_game_rosters` `_game_info`	`nhl_schedules` …
`load_nhl_scoring` `_penalties` `_three_stars` `_scratches`	`nhl_scoring` …
`load_nhl_linescore` `_shifts` `_officials` `_shots_by_period` `_shootout`	`nhl_shifts` …
`load_nhl_groups` `_group_seasons` `_group_aliases` `_team_group_seasons`	`nhl_groups`: conference / division by season, 1918+

sdv-py aliases: `load_nhl_schedules()`, `load_nhl_{player,team,skater,goalie}_boxscores()`.

```
# SQLite mirror of NHL pbp (DBI + RSQLite)
update_nhl_db(dbdir = ".",
  dbname = "fastRhockey_db",
  tblname = "fastRhockey_nhl_pbp",
  force_rebuild = FALSE)
```

NHL WEB API · NHL_*

GAMES & PLAY-BY-PLAY

```
feed <- nhl_game_feed(2023020001)
  # list: pbp, game_info, rosters
pbp  <- nhl_game_pbp(id,
  include_shifts = TRUE)
box  <- nhl_game_boxscore(2023020001)
  # game_info, team_box, skater/goalie_stats
nhl_game_shifts(id, detailed = TRUE)
  # one row per player-shift
nhl_gamecenter_landing(id)
nhl_gamecenter_right_rail(id)
```

Also `nhl_game_story / _content`, `nhl_wsc_pbp`, `nhl_ppt_replay / _goal`. `raw = TRUE` returns the JSON list.

SCHEDULE, SCORES, STANDINGS

```
nhl_schedule(day = "2024-01-15")
nhl_schedule(season = 20232024,
  team_abbr = "TOR", game_type = "regular")
nhl_schedule_calendar(date)
nhl_club_schedule("BOS", season)
nhl_scoreboard(date); nhl_scores(date)
nhl_tv_schedule(date)
nhl_standings(date = "2024-03-01")
nhl_standings_season()
nhl_seasons(); nhl_playoff_bracket(2024)
```

Also `nhl_playoff_carousel / _schedule`, `nhl_where_to_watch()`, `nhl_partner_game_odds("US")`.

TEAMS, ROSTERS, PLAYERS

```
nhl_teams(season); nhl_teams_info("TOR")
nhl_teams_roster("TOR", season = 20232024)
nhl_teams_stats("TOR", season, game_type = 2)
nhl_roster_season("TOR")
nhl_player_info(8478402)   # McDavid
nhl_player_game_log(id, season, game_type = 2)
nhl_player_stats(id); nhl_player_spotlight()
```

Also `nhl_team_prospects()`, `nhl_club_stats_season()`, `nhl_team_scoreboard()`, `nhl_conferences()` / `nhl_divisions()` + `_info` variants.

Draft

`nhl_draft()` · `nhl_draft_year(year, round)` · `nhl_draft_prospects()` / `_info()` · `nhl_draft_rankings(season, category)` · `nhl_draft_tracker()`.

SEASON STATS & LEADERS

```
nhl_stats_skaters(report_type = "summary",
  season = 20232024, game_type = 2)
nhl_stats_goalies(...); nhl_stats_teams(...)
nhl_skater_stats_leaders(season,
  categories = "points")
nhl_goalie_stats_leaders(season)
nhl_stats_draft(year); nhl_stats_seasons()
```

`report_type`: `summary bios realtime penalties powerplay penaltykill scoringRates` …; page with `limit` / `start` / `sort`.

Aggregators (multi-call helpers)

`nhl_game_ids_by_season()` · `nhl_all_players_by_season()` · `nhl_player_career_stats()` · `nhl_team / _skater / _goalie_summary_range()`.

STATS REST (API.NHLE.COM/STATS/REST)

```
nhl_stats_misc(endpoint = "glossary")
nhl_stats_players(limit = 100)
nhl_stats_skater_leaders("points")
nhl_stats_franchise(); nhl_stats_country()
nhl_stats_team_listing()
```

Also `nhl_stats_goalie_leaders` · `_game_listing` · `_skater/_goalie_milestones` · `_glossary` · `_config` · `_ping` · `_content_module`. Cayenne filters: `cayenne_exp = "seasonId=20232024"`.

NHL EDGE TRACKING

```
nhl_edge_skater_detail(8478402, season)
nhl_edge_skater_landing(season)
nhl_edge_skater_comparison(id)
nhl_edge_goalie_detail(id)
nhl_edge_team_detail(team_id)
```

35 wrappers: `nhl_edge_skater_* / _goalie_* / _team_*` (`_detail` + `_top_10`) + `nhl_cat_edge_*_detail`.

RECORDS (RECORDS.NHL.COM)

```
nhl_records_franchise(franchise_id = 10)
nhl_records_player_stats(
  cayenne_exp = "playerId=8478402")
nhl_records_draft(); nhl_records_trophy()
nhl_records_hof_players()
```

25 wrappers: `nhl_records_franchise_* _player* _skater_* _goalie_* _draft_*`, awards, HOF, officials, attendance, venue, combine.

ESPN NHL (ESPN_NHL_*, 126 FNS)

```
espn_nhl_scoreboard(dates = 20240115)
espn_nhl_schedule(team_id = 21, season = 2024)
espn_nhl_teams()
espn_nhl_team_roster(team_id = 21)
espn_nhl_pbp(game_id = 401559593)
espn_nhl_player_box(game_id)
espn_nhl_team_box(game_id)
espn_nhl_summary(id)
espn_nhl_standings(season = 2024)
espn_nhl_athletes_index(active = TRUE,
  limit = 100)
```

Core-API families: `espn_nhl_game_*` · `espn_nhl_player_*` · `espn_nhl_season_*` · `espn_nhl_team_*` + leaders, draft, news, venues, franchises, coaches. College hockey: `espn_mch_*` / `espn_wch_*` (121 each, p. 2).

Fox Sports (Bifrost, read-only)

`fox_nhl_pbp / _boxscore / _odds / _team_roster / _team_stats / _team_gamelog / _standings / _league_leaders`.

EXPECTED GOALS (NHL_XG)

```
pbp <- nhl_game_pbp(2023020001)
pbp <- helper_nhl_calculate_xg(pbp)
  # adds xg column
refresh_xg_models()      # re-download models
```

Three XGBoost models (5v5, special teams, penalty shot) from `fastRhockey-nhl-data`, cached once. Needs suggested `xgboost`; without it `xg` is `NA`.

HELPERS & BUNDLED DATA

```
most_recent_nhl_season()  # 2025 (end year)
most_recent_nhl_season_api_param()  # 20242025
nhl_team_logos  # 34 x 11 tibble
```

`nhl_team_logos`: abbrs, colors, ESPN logos; `division != "Relocated"` = 32 active clubs. Also `pwhl_team_logos` / `phf_team_logos` (p. 2). Proxy for every request: `options(fastRhockey.proxy = "http://host:port")`.

PYTHON · SPORTSDATAVERSE.NHL

`load_nhl_pbp / _schedule / _player_boxscore / _team_boxscore` (same release tags) · live `nhl_web_*` `nhl_edge_*` `nhl_stats_rest_*` `nhl_records_*` `espn_nhl_*` `fox_nhl_*` · models `nhl_goalie_gsax()` `nhl_skater_rapm()` `nhl_skater_war()`.

Beyond the NHL :: PWHL · HockeyTech · Europe

Python mirror: sportsdataverse.pwhl · sportsdataverse.hockey.<league> · data: fastRhockey-pwhl-raw / -pwhl-data · see also: hockeyR (NHL, non-SDV)

PWHL · PWHL_* (HOCKEYTECH FEEDS)

GAMES & PLAY-BY-PLAY

```
library(fastRhockey)
sch <- pwhl_schedule(season = 2025,
  game_type = "both")
pwhl_scorebar(days_back = 3, days_ahead = 3)
pwhl_game_info(game_id = 1)
gs  <- pwhl_game_summary(1) # rosters, 3 stars
pbp <- pwhl_pbp(1)  # one row per event
box <- pwhl_player_box(1)  # per-game box
```

`season` is the concluding year of XXXX-YY (2025 = 2024-25; 2024 = inaugural). `game_type` : `"regular"` · `"playoffs"` · `"both"` . Feeds: `statviewfeed` , `modulekit` , `gc` (JSONP stripped internally).

TEAMS, PLAYERS, LEAGUE

```
pwhl_teams(); pwhl_standings(season = 2025)
pwhl_team_roster(team = "Toronto",
  season = 2025)
pwhl_player_info(id); pwhl_player_stats(id)
pwhl_player_game_log(player_id, season = 2025,
                     game_type = "both")
pwhl_player_search("Poulin")
pwhl_stats(position = "skater", team = "all",
           season = 2025, regular = TRUE)
pwhl_leaders(position = "skaters",
  season = 2025,
             game_type = "regular",
  limit = 100)
pwhl_streaks(season)
pwhl_transactions(season)
pwhl_playoff_bracket(season)
pwhl_season_id(season)
```

`pwhl_stats(position = "goalie")` for netminders (`team` = abbr like `"BOS"`). `pwhl_team_roster(team = )` matches the `team_label` from `pwhl_teams()` . `pwhl_season_id()` returns the HockeyTech season table (id, name, `season_yr`).

ON-ICE ANALYTICS

```
shifts <- pwhl_game_shifts(game_id)
  # one row per stint
toi    <- pwhl_player_toi(game_id)
  # TOI per player
corsi  <- pwhl_game_corsi(game_id)
  # CF/CA, FF/FA
```

Same trio exists for every HockeyTech league: `ahl_game_shifts()` , `ohl_player_toi()` , `whl_game_corsi()` …

PHF (CEASED)

Every `phf_*` / `load_phf_*` call and `update_phf_db()` raises `lifecycle::deprecate_stop()` ; `phf_team_logos` stays. History: **fastRhockey-data** releases; the PWHL is the successor surface.

PWHL RELEASE LOADERS (LOAD_PWHL_*)

Full seasons from the **sportsdataverse-data** releases (tags `pwhl_*`); default `seasons = most_recent_pwhl_season()` ; `dbConnection` / `tablename` optional.

```
pbp <- load_pwhl_pbp(seasons = 2024:2025)
load_pwhl_player_box(2025)
load_pwhl_team_box(2025)
load_pwhl_skater_box(2025)
load_pwhl_goalie_box(2025)
load_pwhl_schedule(2025)
load_pwhl_rosters(2025)
load_pwhl_game_rosters(2025)
load_pwhl_game_info(2025)
```

Loader	Release tag
`load_pwhl_pbp`	`pwhl_pbp`
`load_pwhl_player_box` `_skater_box` `_goalie_box` `_team_box`	`pwhl_*_boxscores`
`load_pwhl_schedule` `_rosters` `_game_rosters` `_game_info`	`pwhl_schedules` …
`load_pwhl_scoring_summary` `_penalty_summary` `_three_stars` `_officials`	`pwhl_scoring_summary` …
`load_pwhl_shots_by_period` `_shootout`	`pwhl_shots_by_period` · `pwhl_shootout`
`load_pwhl_shifts` `_xg_pbp`	`pwhl_shifts` · `pwhl_xg_pbp`

Python-parity aliases: `load_pwhl_schedules()` `load_pwhl_player_boxscores()` `load_pwhl_skater_boxscores()` `load_pwhl_goalie_boxscores()` `load_pwhl_team_boxscores()` .

```
update_pwhl_db(dbdir = ".",
  dbname = "fastRhockey_db",
                force_rebuild = FALSE)
```

BUNDLED DATA

Object	Shape · use
`nhl_team_logos`	34 x 11 — abbr, names, division, colors, ESPN logos
`pwhl_team_logos`	14 x 7 — 6 founders twice (2023-24 + rebrand) + Seattle, Vancouver; join on `team_abbr`
`phf_team_logos`	7 x 7 — historical PHF clubs

PYTHON · SPORTSDATAVERSE

`sportsdataverse.pwhl` / `.hockey.<league>` : the same PWHL, HockeyTech (20 families, adds `echl` `sphl` `ushl` …) and European surface. PWHL extras `pwhl_shot_xg` `pwhl_skater_rapm` `pwhl_team_ratings` `pwhl_in_game_win_prob` .

HOCKEYTECH · AHL OHL WHL QMJHL

ONE SURFACE, FOUR LEAGUES

Each league exports the same 13 callables (thin wrappers over one HockeyTech core, registry in `hockeytech_leagues.R`). Swap the prefix; the signatures match.

```
ahl_season_id()  # seasons + ids
ahl_schedule(season = 2025)  # or season_id =
ahl_standings(season = 2025)
ahl_teams(season = 2025)
ahl_team_roster(team_id, season = 2025)
ahl_leaders(season = 2025)
ahl_player_stats(player_id)
ahl_pbp(game_id); ahl_game_summary(game_id)
ahl_game_shifts(game_id)
ahl_player_toi(game_id)
ahl_game_corsi(game_id)
most_recent_ahl_season()
```

League	Prefix	Notes
PWHL	`pwhl_`	richer surface (col 1)
AHL	`ahl_`	league_id 4
OHL	`ohl_`	league_id 1
WHL	`whl_`	league_id 7
QMJHL	`qmjhl_`	league_id 6; client `lhjmq` , leaguestat host

`season` = end year resolved to a HockeyTech `season_id` via the modulekit seasons feed; pass `season_id` directly to skip the lookup. Override any league key with env `SDV_<LEAGUE>_API_KEY` (e.g. `SDV_AHL_API_KEY`); defaults ship in the package.

`*_pbp()` `x_coord` / `y_coord` : one 600 x 300 canvas, centre ice (300, 150), home attacks x = 0; `_fixed` / `_right` / `_vertical` columns are rotations in feet.

Overviews: `?ahl` `?ohl` `?whl` `?qmjhl` . HockeyTech errors arrive as HTTP 200 with an error body; wrappers return empty frames gracefully.

SEASON HELPERS

`most_recent_{nhl,pwhl,ahl,ohl,whl,qmjhl,phf}_season()` return the season END year; `most_recent_nhl_season_api_param()` = `YYYYYYYY` . PWHL spans Nov–May, so Jun–Oct resolves to the just-finished season.

SPORTSDATAVERSE SIBLINGS

hoopR / wehoop · cfbfastR · baseballr · oddsapiR · sportyR (rinks) · sportsdataverse-R; data: `fastRhockey-data` , `-nhl-data` , `-pwhl-data` , `sportsdataverse-data` .

EUROPE · COLLEGE · 453 WRAPPERS

EUROPEAN LEAGUES (211 FNS)

```
del_schedule(season = 2027, league_id = 1)
del_pbp(match_id = 4444)
shl_schedule(season = 2026)
shl_pbp(game_uuid = "bdhvuc5tex")
khl_schedule(season = 2026)
liiga_schedule(season = 2026,
  tournament = "runkosarja")
sihf_results(season = 2026, league_id = 1)
nl_schedule(season = 2026)
vhl_schedule(season = 2026, stage = "playoff")
```

League	Prefix · fns
DEL (Germany)	`del_` 12
ICE Hockey League (Austria)	`icehl_` 12
EIHL (UK)	`eihl_` 9
Metal Ligaen (Denmark)	`metal_` 8
KHL	`khl_` 23
VHL (Russia)	`vhl_` 14
SHL (Sweden)	`shl_` 29
HockeyAllsvenskan	`ha_` 11
Liiga (Finland)	`liiga_` 23
SIHF (Swiss federation)	`sihf_` 33
National League (Swiss)	`nl_` 17
Extraliga (Czechia)	`elh_` 9
Ligue Magnus (France)	`magnus_` 11

Seasons are END years (2027 = 2026-27). At least 1 s between requests to a host; 429 / 5xx retried honouring `Retry-After` . `return_parsed = FALSE` returns the raw payload. `nl_*` : polite, incremental use only (see its help).

ESPN COLLEGE HOCKEY

```
espn_mch_scoreboard(dates = "20250110")
espn_wch_pbp(game_id = "401762970")
espn_mch_standings(season = 2025)
```

121 wrappers per league, `espn_mch_*` (men) and `espn_wch_*` (women), mirroring `espn_nhl_*` incl. `_pbp` / `_team_box` / `_player_box` ; defaults match sportsdataverse-py.

Classed errors (Europe + college)

```
tryCatch(khl_schedule(season = 2026),
  fastRhockey_no_data = function(e) NULL)
```

`fastRhockey_no_data` (nothing there) · `fastRhockey_asset_fetch` (fetch failed, answer unknown) · `fastRhockey_bad_request` ; all inherit `fastRhockey_error` .

fastRhockey :: CHEAT SHEET

Tidy R access to hockey play-by-play, box scores, schedules, rosters and stats: the NHL Web API (api-web.nhle.com), NHL Stats REST, NHL EDGE tracking, NHL Records, ESPN and Fox NHL feeds, the PWHL plus AHL / OHL / WHL / QMJHL (HockeyTech), 13 European leagues, ESPN men's and women's college hockey, and season loaders from the SportsDataverse data releases: 880 exports. Sister of **hoopR** / **wehoop** / **cfbfastR**; Python mirror: `sportsdataverse.nhl` / `.pwhl` / `.hockey`. Page 1 of 2: NHL. Page 2: PWHL, HockeyTech, Europe, college, PHF.

v1.0.0.9000
R >= 4.1.0 · httr2 · dplyr · data.table
fastRhockey.sportsdataverse.org

GET STARTED

```
install.packages("fastRhockey")
# dev build
remotes::install_github(
  "sportsdataverse/fastRhockey")
library(fastRhockey)
library(dplyr)
```

Every function returns a tibble of class `fastRhockey_data` (or a named list of them). Live NHL wrappers hit `api-web.nhle.com/v1/` with no API key. Endpoint family overviews: `?nhl_web` `?nhl_stats_rest` `?nhl_edge` `?nhl_records` `?nhl_loaders` `?nhl_xg` `?pwhl`.

NHL RELEASE LOADERS

Pre-scraped full seasons (2010+) from the **sportsdataverse-data** GitHub releases (tag `nhl_*`). Default `seasons = most_recent_nhl_season()`; optional `dbConnection`, `tablename` write straight to a DB.

```
pbp  <- load_nhl_pbp(seasons = 2023:2025)
lite <- load_nhl_pbp_lite(2025)
  # slim columns
tb   <- load_nhl_team_box(2025)
pb   <- load_nhl_player_box(2025)
  # + skater / goalie
sch  <- load_nhl_schedule(2020:2025)
```

Loader	Release tag
`load_nhl_pbp` · `_pbp_full` · `_pbp_lite`	`nhl_pbp_full` / `nhl_pbp_lite`
`load_nhl_team_box` `_player_box` `_skater_box` `_goalie_box`	`nhl_*_boxscores`
`load_nhl_schedule` `_rosters` `_game_rosters` `_game_info`	`nhl_schedules` …
`load_nhl_scoring` `_penalties` `_three_stars` `_scratches`	`nhl_scoring` …
`load_nhl_linescore` `_shifts` `_officials` `_shots_by_period` `_shootout`	`nhl_shifts` …
`load_nhl_groups` `_group_seasons` `_group_aliases` `_team_group_seasons`	`nhl_groups`: conference / division by season, 1918+

sdv-py aliases: `load_nhl_schedules()`, `load_nhl_{player,team,skater,goalie}_boxscores()`.

```
# SQLite mirror of NHL pbp (DBI + RSQLite)
update_nhl_db(dbdir = ".",
  dbname = "fastRhockey_db",
  tblname = "fastRhockey_nhl_pbp",
  force_rebuild = FALSE)
```

NHL WEB API · NHL_*

GAMES & PLAY-BY-PLAY

```
feed <- nhl_game_feed(2023020001)
  # list: pbp, game_info, rosters
pbp  <- nhl_game_pbp(id,
  include_shifts = TRUE)
box  <- nhl_game_boxscore(2023020001)
  # game_info, team_box, skater/goalie_stats
nhl_game_shifts(id, detailed = TRUE)
  # one row per player-shift
nhl_gamecenter_landing(id)
nhl_gamecenter_right_rail(id)
```

Also `nhl_game_story / _content`, `nhl_wsc_pbp`, `nhl_ppt_replay / _goal`. `raw = TRUE` returns the JSON list.

SCHEDULE, SCORES, STANDINGS

```
nhl_schedule(day = "2024-01-15")
nhl_schedule(season = 20232024,
  team_abbr = "TOR", game_type = "regular")
nhl_schedule_calendar(date)
nhl_club_schedule("BOS", season)
nhl_scoreboard(date); nhl_scores(date)
nhl_tv_schedule(date)
nhl_standings(date = "2024-03-01")
nhl_standings_season()
nhl_seasons(); nhl_playoff_bracket(2024)
```

Also `nhl_playoff_carousel / _schedule`, `nhl_where_to_watch()`, `nhl_partner_game_odds("US")`.

TEAMS, ROSTERS, PLAYERS

```
nhl_teams(season); nhl_teams_info("TOR")
nhl_teams_roster("TOR", season = 20232024)
nhl_teams_stats("TOR", season, game_type = 2)
nhl_roster_season("TOR")
nhl_player_info(8478402)   # McDavid
nhl_player_game_log(id, season, game_type = 2)
nhl_player_stats(id); nhl_player_spotlight()
```

Also `nhl_team_prospects()`, `nhl_club_stats_season()`, `nhl_team_scoreboard()`, `nhl_conferences()` / `nhl_divisions()` + `_info` variants.

Draft

`nhl_draft()` · `nhl_draft_year(year, round)` · `nhl_draft_prospects()` / `_info()` · `nhl_draft_rankings(season, category)` · `nhl_draft_tracker()`.

SEASON STATS & LEADERS

```
nhl_stats_skaters(report_type = "summary",
  season = 20232024, game_type = 2)
nhl_stats_goalies(...); nhl_stats_teams(...)
nhl_skater_stats_leaders(season,
  categories = "points")
nhl_goalie_stats_leaders(season)
nhl_stats_draft(year); nhl_stats_seasons()
```

`report_type`: `summary bios realtime penalties powerplay penaltykill scoringRates` …; page with `limit` / `start` / `sort`.

Aggregators (multi-call helpers)

`nhl_game_ids_by_season()` · `nhl_all_players_by_season()` · `nhl_player_career_stats()` · `nhl_team / _skater / _goalie_summary_range()`.

STATS REST (API.NHLE.COM/STATS/REST)

```
nhl_stats_misc(endpoint = "glossary")
nhl_stats_players(limit = 100)
nhl_stats_skater_leaders("points")
nhl_stats_franchise(); nhl_stats_country()
nhl_stats_team_listing()
```

Also `nhl_stats_goalie_leaders` · `_game_listing` · `_skater/_goalie_milestones` · `_glossary` · `_config` · `_ping` · `_content_module`. Cayenne filters: `cayenne_exp = "seasonId=20232024"`.

NHL EDGE TRACKING

```
nhl_edge_skater_detail(8478402, season)
nhl_edge_skater_landing(season)
nhl_edge_skater_comparison(id)
nhl_edge_goalie_detail(id)
nhl_edge_team_detail(team_id)
```

35 wrappers: `nhl_edge_skater_* / _goalie_* / _team_* (_detail + _top_10)` + `nhl_cat_edge_*_detail`.

RECORDS (RECORDS.NHL.COM)

```
nhl_records_franchise(franchise_id = 10)
nhl_records_player_stats(
  cayenne_exp = "playerId=8478402")
nhl_records_draft(); nhl_records_trophy()
nhl_records_hof_players()
```

25 wrappers: `nhl_records_franchise_* _player* _skater_* _goalie_* _draft_*`, awards, HOF, officials, attendance, venue, combine.

ESPN NHL (ESPN_NHL_*, 126 FNS)

```
espn_nhl_scoreboard(dates = 20240115)
espn_nhl_schedule(team_id = 21, season = 2024)
espn_nhl_teams()
espn_nhl_team_roster(team_id = 21)
espn_nhl_pbp(game_id = 401559593)
espn_nhl_player_box(game_id)
espn_nhl_team_box(game_id)
espn_nhl_summary(id)
espn_nhl_standings(season = 2024)
espn_nhl_athletes_index(active = TRUE,
  limit = 100)
```

Core-API families: `espn_nhl_game_*` · `espn_nhl_player_*` · `espn_nhl_season_*` · `espn_nhl_team_*` + leaders, draft, news, venues, franchises, coaches. College hockey: `espn_mch_*` / `espn_wch_*` (121 each, p. 2).

Fox Sports (Bifrost, read-only)

`fox_nhl_pbp / _boxscore / _odds / _team_roster / _team_stats / _team_gamelog / _standings / _league_leaders`.

EXPECTED GOALS (NHL_XG)

```
pbp <- nhl_game_pbp(2023020001)
pbp <- helper_nhl_calculate_xg(pbp)
  # adds xg column
refresh_xg_models()      # re-download models
```

Three XGBoost models (5v5, special teams, penalty shot) from `fastRhockey-nhl-data`, cached once. Needs suggested `xgboost`; without it `xg` is `NA`.

HELPERS & BUNDLED DATA

```
most_recent_nhl_season()  # 2025 (end year)
most_recent_nhl_season_api_param()  # 20242025
nhl_team_logos  # 34 x 11 tibble
```

`nhl_team_logos`: abbrs, colors, ESPN logos; `division != "Relocated"` = 32 active clubs. Also `pwhl_team_logos` / `phf_team_logos` (p. 2). Proxy for every request: `options(fastRhockey.proxy = "http://host:port")`.

PYTHON · SPORTSDATAVERSE.NHL

`load_nhl_pbp / _schedule / _player_boxscore / _team_boxscore` (same release tags) · live `nhl_web_*` `nhl_edge_*` `nhl_stats_rest_*` `nhl_records_*` `espn_nhl_*` `fox_nhl_*` · models `nhl_goalie_gsax()` `nhl_skater_rapm()` `nhl_skater_war()`.

Beyond the NHL :: PWHL · HockeyTech · Europe

Python mirror: **sportsdataverse.pwhl** · **sportsdataverse.hockey.<league>** · data: **fastRhockey-pwhl-raw** / **-pwhl-data** · see also: **hockeyR** (NHL, non-SDV)

PWHL · PWHL_* (HOCKEYTECH FEEDS)

GAMES & PLAY-BY-PLAY

```
library(fastRhockey)
sch <- pwhl_schedule(season = 2025,
  game_type = "both")
pwhl_scorebar(days_back = 3, days_ahead = 3)
pwhl_game_info(game_id = 1)
gs  <- pwhl_game_summary(1) # rosters, 3 stars
pbp <- pwhl_pbp(1)  # one row per event
box <- pwhl_player_box(1)  # per-game box
```

`season` is the concluding year of XXXX-YY (2025 = 2024-25; 2024 = inaugural). `game_type`: `"regular"` · `"playoffs"` · `"both"`. Feeds: `statviewfeed`, `modulekit`, `gc` (JSONP stripped internally).

TEAMS, PLAYERS, LEAGUE

```
pwhl_teams(); pwhl_standings(season = 2025)
pwhl_team_roster(team = "Toronto",
  season = 2025)
pwhl_player_info(id); pwhl_player_stats(id)
pwhl_player_game_log(player_id, season = 2025,
                     game_type = "both")
pwhl_player_search("Poulin")
pwhl_stats(position = "skater", team = "all",
           season = 2025, regular = TRUE)
pwhl_leaders(position = "skaters",
  season = 2025,
             game_type = "regular",
  limit = 100)
pwhl_streaks(season)
pwhl_transactions(season)
pwhl_playoff_bracket(season)
pwhl_season_id(season)
```

`pwhl_stats(position = "goalie")` for netminders (`team` = abbr like `"BOS"`). `pwhl_team_roster(team = )` matches the `team_label` from `pwhl_teams()`. `pwhl_season_id()` returns the HockeyTech season table (id, name, `season_yr`).

ON-ICE ANALYTICS

```
shifts <- pwhl_game_shifts(game_id)
  # one row per stint
toi    <- pwhl_player_toi(game_id)
  # TOI per player
corsi  <- pwhl_game_corsi(game_id)
  # CF/CA, FF/FA
```

Same trio exists for every HockeyTech league: `ahl_game_shifts()`, `ohl_player_toi()`, `whl_game_corsi()` …

PHF (CEASED)

Every `phf_*` / `load_phf_*` call and `update_phf_db()` raises `lifecycle::deprecate_stop()`; `phf_team_logos` stays. History: **fastRhockey-data** releases; the PWHL is the successor surface.

PWHL RELEASE LOADERS (LOAD_PWHL_*)

Full seasons from the **sportsdataverse-data** releases (tags `pwhl_*`); default `seasons = most_recent_pwhl_season()`; `dbConnection` / `tablename` optional.

```
pbp <- load_pwhl_pbp(seasons = 2024:2025)
load_pwhl_player_box(2025)
load_pwhl_team_box(2025)
load_pwhl_skater_box(2025)
load_pwhl_goalie_box(2025)
load_pwhl_schedule(2025)
load_pwhl_rosters(2025)
load_pwhl_game_rosters(2025)
load_pwhl_game_info(2025)
```

Loader	Release tag
`load_pwhl_pbp`	`pwhl_pbp`
`load_pwhl_player_box` `_skater_box` `_goalie_box` `_team_box`	`pwhl_*_boxscores`
`load_pwhl_schedule` `_rosters` `_game_rosters` `_game_info`	`pwhl_schedules` …
`load_pwhl_scoring_summary` `_penalty_summary` `_three_stars` `_officials`	`pwhl_scoring_summary` …
`load_pwhl_shots_by_period` `_shootout`	`pwhl_shots_by_period` · `pwhl_shootout`
`load_pwhl_shifts` `_xg_pbp`	`pwhl_shifts` · `pwhl_xg_pbp`

Python-parity aliases: `load_pwhl_schedules()` `load_pwhl_player_boxscores()` `load_pwhl_skater_boxscores()` `load_pwhl_goalie_boxscores()` `load_pwhl_team_boxscores()`.

```
update_pwhl_db(dbdir = ".",
  dbname = "fastRhockey_db",
                force_rebuild = FALSE)
```

BUNDLED DATA

Object	Shape · use
`nhl_team_logos`	34 x 11 — abbr, names, division, colors, ESPN logos
`pwhl_team_logos`	14 x 7 — 6 founders twice (2023-24 + rebrand) + Seattle, Vancouver; join on `team_abbr`
`phf_team_logos`	7 x 7 — historical PHF clubs

PYTHON · SPORTSDATAVERSE

`sportsdataverse.pwhl` / `.hockey.<league>`: the same PWHL, HockeyTech (20 families, adds `echl sphl ushl` …) and European surface. PWHL extras `pwhl_shot_xg` `pwhl_skater_rapm` `pwhl_team_ratings` `pwhl_in_game_win_prob`.

HOCKEYTECH · AHL OHL WHL QMJHL

ONE SURFACE, FOUR LEAGUES

Each league exports the same 13 callables (thin wrappers over one HockeyTech core, registry in `hockeytech_leagues.R`). Swap the prefix; the signatures match.

```
ahl_season_id()  # seasons + ids
ahl_schedule(season = 2025)  # or season_id =
ahl_standings(season = 2025)
ahl_teams(season = 2025)
ahl_team_roster(team_id, season = 2025)
ahl_leaders(season = 2025)
ahl_player_stats(player_id)
ahl_pbp(game_id); ahl_game_summary(game_id)
ahl_game_shifts(game_id)
ahl_player_toi(game_id)
ahl_game_corsi(game_id)
most_recent_ahl_season()
```

League	Prefix	Notes
PWHL	`pwhl_`	richer surface (col 1)
AHL	`ahl_`	league_id 4
OHL	`ohl_`	league_id 1
WHL	`whl_`	league_id 7
QMJHL	`qmjhl_`	league_id 6; client `lhjmq`, leaguestat host

`season` = end year resolved to a HockeyTech `season_id` via the modulekit seasons feed; pass `season_id` directly to skip the lookup. Override any league key with env `SDV_<LEAGUE>_API_KEY` (e.g. `SDV_AHL_API_KEY`); defaults ship in the package.

`*_pbp()` `x_coord` / `y_coord`: one 600 x 300 canvas, centre ice (300, 150), home attacks x = 0; `_fixed` / `_right` / `_vertical` columns are rotations in feet.

Overviews: `?ahl` `?ohl` `?whl` `?qmjhl`. HockeyTech errors arrive as HTTP 200 with an error body; wrappers return empty frames gracefully.

SEASON HELPERS

`most_recent_{nhl,pwhl,ahl,ohl,whl,qmjhl,phf}_season()` return the season END year; `most_recent_nhl_season_api_param()` = `YYYYYYYY`. PWHL spans Nov–May, so Jun–Oct resolves to the just-finished season.

SPORTSDATAVERSE SIBLINGS

hoopR / **wehoop** · **cfbfastR** · **baseballr** · **oddsapiR** · **sportyR** (rinks) · **sportsdataverse-R**; data: `fastRhockey-data`, `-nhl-data`, `-pwhl-data`, `sportsdataverse-data`.

EUROPE · COLLEGE · 453 WRAPPERS

EUROPEAN LEAGUES (211 FNS)

```
del_schedule(season = 2027, league_id = 1)
del_pbp(match_id = 4444)
shl_schedule(season = 2026)
shl_pbp(game_uuid = "bdhvuc5tex")
khl_schedule(season = 2026)
liiga_schedule(season = 2026,
  tournament = "runkosarja")
sihf_results(season = 2026, league_id = 1)
nl_schedule(season = 2026)
vhl_schedule(season = 2026, stage = "playoff")
```

League	Prefix · fns
DEL (Germany)	`del_` 12
ICE Hockey League (Austria)	`icehl_` 12
EIHL (UK)	`eihl_` 9
Metal Ligaen (Denmark)	`metal_` 8
KHL	`khl_` 23
VHL (Russia)	`vhl_` 14
SHL (Sweden)	`shl_` 29
HockeyAllsvenskan	`ha_` 11
Liiga (Finland)	`liiga_` 23
SIHF (Swiss federation)	`sihf_` 33
National League (Swiss)	`nl_` 17
Extraliga (Czechia)	`elh_` 9
Ligue Magnus (France)	`magnus_` 11

Seasons are END years (2027 = 2026-27). At least 1 s between requests to a host; 429 / 5xx retried honouring `Retry-After`. `return_parsed = FALSE` returns the raw payload. `nl_*`: polite, incremental use only (see its help).

ESPN COLLEGE HOCKEY

```
espn_mch_scoreboard(dates = "20250110")
espn_wch_pbp(game_id = "401762970")
espn_mch_standings(season = 2025)
```

121 wrappers per league, `espn_mch_*` (men) and `espn_wch_*` (women), mirroring `espn_nhl_*` incl. `_pbp` / `_team_box` / `_player_box`; defaults match sportsdataverse-py.

Classed errors (Europe + college)

```
tryCatch(khl_schedule(season = 2026),
  fastRhockey_no_data = function(e) NULL)
```

`fastRhockey_no_data` (nothing there) · `fastRhockey_asset_fetch` (fetch failed, answer unknown) · `fastRhockey_bad_request`; all inherit `fastRhockey_error`.